



GreenID - AI-Powered Green Activity Verification

Project Summary & Interview Preparation Guide

1. Project Overview & Architecture

GreenID is a comprehensive platform designed to encourage and verify environmentally friendly activities. Users can log their green contributions and receive automated verification powered by Machine Learning and mathematical algorithms.

Tech Stack

- **Frontend (UI):** React.js (Vite), Tailwind CSS
- **Backend (API & DB):** Node.js, Express.js, MongoDB (Mongoose)
- **Machine Learning Service:** Python, Flask, TensorFlow, Keras, OpenCV
- **AI Integration:** Google Generative AI (Gemini Chatbot - "HariBaba")
- **Cloud Storage:** Cloudinary, Multer

Core ML Features & Algorithms

1. **Walk Verification (Haversine Formula):** Uses the Haversine formula to compute the great-circle distance between a series of GPS coordinates to verify if the user has walked a sufficient distance (threshold: 2.0 km).
 2. **Public Transport Verification (MobileNetV2):** Uses a lightweight Deep Learning image classification model (`tf.keras`) to predict if the user is in an auto-rickshaw, bus, or metro. Requires a minimum confidence score of 60%.
 3. **Tree Planting Verification (Computer Vision/OpenCV):** Extracts frames from an uploaded video (sampled at 1 fps), checking for motion, person, and plant components to compute a verification confidence score.
 4. **Cleanup Drive Verification:** Leverages an ML-based image comparison utility to evaluate "before" and "after" images, verifying that the location was successfully cleaned based on a confidence threshold.
-



2. Technical Decisions & Algorithm Justifications

Q1: Why did you separate the Machine Learning backend (Flask) from the Main backend (Node.js)?

A: Python is the industry standard for Data Science and Machine Learning (TensorFlow, OpenCV, Scikit-learn), whereas Node.js excels at high-concurrency, asynchronous I/O, and REST APIs. By using a microservices-like architecture, each service operates in its optimal environment. ML tasks are CPU/GPU heavy and blocking, which would severely degrade Node.js performance if integrated directly.

Q2: Why did you use MobileNetV2 for Image Classification instead of larger models like ResNet or VGG16?

A: MobileNetV2 is highly optimized for performance, utilizing depthwise separable convolutions that drastically reduce the number of parameters and computational cost. Since GreenID requires relatively quick real-time inferences for public transport verification—and to minimize server overhead—MobileNetV2 provides an optimal balance between accuracy, model size, and latency.

Q3: What is the Haversine formula, and why didn't you just use standard Euclidean distance for Walk Verification?

A: The Haversine formula calculates the shortest distance between two points on the surface of a sphere based on their latitudes and longitudes. Because the Earth is roughly spherical, using straight-line Euclidean distance (Pythagorean theorem) will result in inaccurate distance calculations over the Earth's curvature. Haversine gives us an accurate real-world walking distance in kilometers.

Q4: How do you handle video processing for Tree Planting without crashing the server?

A: Video processing is resource-intensive. Instead of loading the entire video into memory, the Flask service saves the file temporarily to disk. It then uses OpenCV to sample frames at a low frame rate (1 fps) up to a maximum limit (e.g., 60 frames). It processes these individual frames to detect motion and objects. Afterwards, temporary files and frames are strictly cleaned up via a cleanup utility to prevent memory leaks and disk space overflow.

3. MERN Stack & General Web Dev Questions

Q5: How does your application handle Security and Authentication?

A: We use JWT (JSON Web Tokens) for secure, stateless authentication. Once a user logs in, the Node.js backend signs a JWT payload and returns it to the client (stored in localStorage or an HttpOnly cookie). For protected routes (like viewing a profile or submitting green activities), the React frontend includes this token in the `Authorization` header. A custom Node middleware verifies the token signature to authenticate the user.

Q6: Why did you choose Vite over Create React App (CRA)?

A: Vite uses native ES modules (`esbuild`), making the local development server start almost instantly and enabling highly responsive Hot Module Replacement (HMR). CRA uses Webpack, which bundles the entire application before serving it, leading to significantly slower startup and reload times as the project grows.

Q7: What is the role of Mongoose, and why not use plain MongoDB drivers?

A: Mongoose is an Object Data Modeling (ODM) library for MongoDB. While plain drivers just dump JSON documents into the database, Mongoose provides a rigorous schema structure. It enforces data types, handles default values, defines validations (like ensuring emails are correctly formatted), and manages relationships between collections cleanly.

Q8: How did you implement CORS (Cross-Origin Resource Sharing) and why is it needed?

A: Browsers enforce a Same-Origin Policy that prevents a frontend app running on `localhost:5173` from making API calls to a backend running on `localhost:5000` (different port/origin). To bypass this securely, we use the `cors` middleware in Express and `flask_cors` in Flask, explicitly whitelisting the frontend's origin via environment variables (e.g., `FRONTEND_URL`) so the browser allows the communication.

Q9: How do you handle File Uploads before passing them to the ML model?

A: In the Node.js backend, we use `multer` to intercept `multipart/form-data` requests. For database persistence, images/videos are uploaded to Cloudinary, providing a secure URL stored in MongoDB. For verification, the files are passed to the Flask service, where `werkzeug.utils.secure_filename` is used to safely save temporary files to the OS's temp directory before inference, protecting against directory traversal attacks.

Q10: How does the HariBaba AI Chatbot work under the hood?

A: We use the `@google/generative-ai` SDK on the backend. When a user sends a message from React, it hits an Express endpoint. The backend constructs a prompt containing strict system instructions telling Gemini to act as a

green living assistant ("HariBaba"). We then proxy the Gemini API response back to the frontend. Handling this on the backend keeps our Gemini API Key safely hidden from the client browser.